

## Midterm 2

### Instructions

- PHY411: Do problems 1–3 (skip 3c)
- PHY 506: Do problems 1-3 (do 3c)

Accept the assignment from github classroom: <https://classroom.github.com/a/uXZdCOKe> . This will create a new repository for you on github, titled something like `compphys-midterm2-username`. You should submit your code through github classroom, and your writeup through UBLearns. If you prefer, you can do your writeup “in-line” in your notebooks (using Markdown cells), convert the notebook to HTML/PDF/etc., and upload the converted notebooks.

If you are using the Docker container, do a `docker pull ucsdcomp/compphys:latest` to pull the latest version, which includes necessary packages like `jax`, `flax`, `tensorflow_datasets`, and `galaxy_datasets` (this is a custom TFDS wrapper for the GalaxyMNIST dataset). If you are not using the Docker container, you can `pip install` the necessary packages into your preferred environment.

Be aware that Problem 3 involved training a CNN, which will take some time (likely 10-30 minutes to train each time, depending on your laptop and the size of your CNN). Plan ahead accordingly, and make sure to save your results (NN params and/or just the results) after each successful training.

## Problem 1 : Pandemic Zombies

*25 points*

**This is intended for educational purposes and is not intended as a realistic simulation of the COVID-19 virus.**

Suppose you have a pandemic of some disease. In this case, we will simulate the zombie apocalypse. As such, we will consider `n_walker` random walkers in 2 dimensions that each take `n_steps` steps. At each step, the walkers can move `dx` units. The walkers are confined to a single 1x1 room with cyclic boundary conditions in x and y (so, the walkers wrap around if they wander off the edge). There is a configurable initial number of infected individuals, `n_infected`.

The starter code below assumes that there is zero rate of transmission from the infected individuals (i.e., passive zombies). The animation shows the paths of the walkers. The healthy individuals show up as blue dots. The zombies show up as red dots.

### Problem 1a

*15 points*

Implement an “infection rate,” `P_i`, and an “infection distance,” `d_i`, where any walkers that are within the infection distance `d_i` to an infected individual have a probability `P_i` to be infected. They then infect others in later time steps. The animation should reflect the changes. (Hint: This is hard to vectorize, so you can just use for loops). Use the following values: `n_walkers=100`, `n_steps=100`, `n_infected=4`, `dx=0.01`, `P_i = 0.1`, and `d_i=0.1`.

### Problem 1b

*10 points*

Plot the number of total infections in the sample as a function of time step for `P_i = 0.1`, `0.2`, `0.3`, and for `d_i = 0.01`, `0.02`, `0.03`. Use the same values as before: `n_walkers=100`, `n_steps=1000`, `n_infected=4`, and `dx=0.01`.

Remember to stay healthy and practice social distancing (keep outside of the rate `d_i` in real life)! (This line has been preserved for historical reference.)

## Problem 2: Geospatial location of COVID19 confirmed cases

*20 points*

This is intended for educational purposes and is not intended as a realistic analysis of the COVID-19 virus confirmed cases.

### Problem 2a

*10 points*

Download the dataset, extract the data for mainland US locations, and plot the data for Erie County. Specifically:

- Execute the following cell to download the data with `wget`.
- Extract the geographical data for mainland US locations, namely the location name (`Combined_Key`), latitude, and longitude from each row. The mainland US can be defined by  $(25 < \text{latitude} < 50)$  and  $(-130 < \text{longitude} < -70)$ .
  - You are welcome to use whatever data structure you like (dict, numpy array, pandas dataframe, etc.).
  - Warning: some automated loaders like `np.genfromtxt` might not work, as some columns have “,” characters inside quotation marks.
- Extract the number of confirmed cases vs. date for each location into another array.
- Plot the number of *new* confirmed cases vs. date for Erie County.
  - The CSV file contains the cumulative number of cases, so you have to compute the number of new cases per day).
  - If you convert the date strings to python `datetime.date` objects, matplotlib can automatically make a nice date-axis for you.

### Problem 2b: Voronoi diagram

*10 points*

Select locations with nonzero confirm cases on March 20, 2020. Make a Voronoi diagram with 10 cells of the geographical coordinates (lat, long) of these locations, i.e., treat the (lat, long) coordinates as if they were 2D Euclidean coordinates. You can adapt the `kmeans.ipynb` notebook, which has been copied to this folder. Specifically

- Assume there are 10 Voronoi cells.
- Initialize the centroids to 10 randomly chosen points from the dataset.
- Run  $k$ -means iterations to compute the centroids that minimize the  $k$ -means distance to the data points.
- For the final configuration, plot:
  - The centroids of the data in black circles.
  - The data points in each cell with a different color.

## Problem 3: Galaxy classification

30 points (PHY411) or 40 points (PHY506)

In this problem, we adapt the image classification notebooks from class to perform image classification on the GalaxyMNIST dataset. We will create and train a few different neural networks and quantify their performance.

The GalaxyMNIST dataset is hosted at <https://github.com/mwalmsley/galaxy-datasets>. It contains 10,000 galaxy images, assigned to four categories:

```
labels = {
    0: "smooth-round",
    1: "smooth-cigar",
    2: "edge-on-disk",
    3: "unbarred-spiral"
}
```

The dataset is a “debugging” subset of a much larger collection of galaxy images (with many more labels), but is sufficient for our purposes. See arXiv:2102.08414, as well as the README in the git repository, for more details on how the dataset was constructed.

### WARNING

Training the neural network will take a fair amount of time: the input images are 224 x 224 pixels, versus 28 x 28 pixels for MNIST, so training will take roughly 100x as long. On my M4 Pro CPU, the first step takes about 15 minutes to train. Therefore, you will have to start early and plan carefully. Some recommendations:

- Use `tqdm` to create a progress bar with time estimate during the training. If the time estimate is too large, you can stop the kernel and revise the parameters to achieve a reasonable training time.
- Each time you obtain a successful training, **save the results!** The Jax MNIST notebook shows how to export the trained weights at the end. See the notebook `restore.ipynb` for how to load the saved parameters.
- Given the computational limitations of running this on a laptop, you will not be graded on absolute accuracy, but rather on following the right training and evaluation methods.

### Dataset loader

For this problem, a `tensorflow_dataset` wrapper has been created to load the dataset in exactly the same way as the JAX MNIST tutorial. The wrapper is in a python package at [https://github.com/ubsuny/galaxy\\_datasets](https://github.com/ubsuny/galaxy_datasets). If you are using the Docker container, this python package has already been installed (run `docker pull ubsuny/compphys:latest` to pull the latest version). Otherwise, you can install it with pip:

```
pip install git+https://github.com/ubsuny/galaxy_datasets@v0.1
```

The first time you run this notebook, the dataset will be downloaded to `data_dir`. If you are using the Docker container, make sure that `data_dir` is somewhere persistent, otherwise it will get deleted every time you stop the container.

### Problem 3a

20 points

Adapt the MNIST tutorial to classify the GalaxyMNIST images. The MNIST tutorial has been copied to this folder. You will need to modify a few things in the training: - Change the sizes of the layers to handle different size of the GalaxyMNIST pictures ( $224 * 224 * 3$  vs.  $28 * 28 * 1$ ). - Note, the 3 reflect the fact that the images are RGB (3 colors) rather than greyscale. - Downscale more aggressively in the pooling layers, to speed up the training (if you had proper hardware like a GPU cluster, you probably wouldn't want to do this!). For the 2 CNN layers, I suggest trying downscaling by a factor of 4 after the first layer and 8 after the second layer. Remember to adjust both `window_shape` and `strides`.

Once you have trained the network, evaluate it on the test dataset. In your writeup, include the following: - Plot of cost function versus training iteration. How do the train and test curves differ, and what do they say about overtraining? - The classification accuracy of the network. - A plot of the confusion matrix, i.e., the  $4 \times 4$  matrix of possible outcomes (4 true categories times 4 classifier categories). Discuss the performance: where does the network perform better or worse? - A few example plots of misclassified galaxies.

### Problem 3b: ROC curve

*10 points*

Now consider your network as a binary classifier for the **unbarred-spiral** category (category 3). In other words, instead of selecting the single category with the highest score, we just care about whether a galaxy is **unbarred-spiral** or not. Specifically, let  $p$  be the probability returned by the network for the **unbarred-spiral** category. For a given threshold  $x \in [0, 1]$ , we classify the image as **unbarred-spiral** if  $p > x$ .

We characterize the performance with two numbers: 1. Signal efficiency: the rate of classifying true unbarred-spiral images as **unbarred\_spiral** (true positives), and 2. Fake rate: the rate of classifying non-**unbarred\_spiral** images as **unbarred\_spiral** (false positives).

The ROC curve is a visualization of the true positive vs. fake positive rate, scanning  $x$  values over the whole range  $[0, 1]$ . See e.g. <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc> for more information.

Create a ROC plot, i.e., plot the signal efficiency (y-axis) vs. fake rate (x-axis) for  $x \in [0, 1]$ . Compute the “area under the curve” (AUC) metric by integrating the curve, and draw the AUC on the plot (remember, the AUC is 1 for a perfect classifier, and 0.5 for random guesses). Upload the ROC plot to your writeup, and discuss the results.

(Comment: the AUC is a popular metric for machine learning competitions. In physics, we are typically more concerned with finding the point on the ROC curve that gives the best signal-vs-background discrimination, e.g.,  $s/\sqrt{b}$ .)

### Problem 3c: improving performance

*10 points*

**PHY506 students only.**

Run a few ( $\approx 3$ ) experiments modifying the network to try to improve the performance. You might want to read through <https://www.kaggle.com/code/cdeotte/how-to-choose-cnn-architecture-mnist> for inspiration. For example, you could try:

- Instead of aggressively downsampling (e.g. by a factor of 8), add a 3rd convolutional layer with pooling.
- Vary the size of the kernel.

- Try a simple multilayer perceptron instead of a CNN.
- Modify the batch size or training rate (but beware blowing up the training time!).

To avoid problems with versioning, variable name conflicts, accidentally reusing old variables, etc..., you might want to create a new notebook for each experiment.

In your writeup, for each experiment, describe what you varied, provide the total classification accuracy (don't worry about the other metrics, unless you are curious), and discuss your findings. (Again, given the limitations of running this on your laptop, you will not be graded on the absolute performance of your NN!)